

Realtime Streaming Image Processing on FPGA

Implementation Report: AXI4-Stream Pipeline, Verification, and Team Contributions



Lukas Roess, Valentin Bumeder, Jan Duchscherer, Justin Loeber

Embedded Systems, Academic Year 2025-2026

1. Abstract

This report documents the implementation status of a real-time image-processing pipeline on Zybo Z7-10 with Pcam 5C input and AXI4-Stream video processing in programmable logic. The integrated processing chain is RGB -> Grayscale -> 3x3 Window -> Sobel -> Threshold/Overlay, verified with cocotb and GHDL using protocol-aware scoreboards and image-model based reference checks.

The current repository contains implemented RTL and verification assets for passthrough, grayscale conversion, window generation, Sobel filtering, filter wrapping, edge overlay, and the button control path (debounce/click detection). The report combines architecture rationale, implementation details, AXI framing guarantees, and simulation evidence extracted from generated JUnit results and output image artifacts.

The project objective is not only feature completeness but deterministic stream behavior under backpressure. Therefore, the verification strategy emphasizes TVALID/TREADY stability, SOF/EOL alignment, and multi-module interoperability at frame boundaries.

Recorded testcases

55

Parsed from JUnit XML artifacts in simulation builds

Passing testcases

55

Current stored artifact set

Module areas covered

6

RTL + control + wrapper scope

The remaining sections provide a module-by-module implementation review and conclude with contribution analytics and prioritized risks.

2. Project Scope and Objectives

The system target is a live camera-to-HDMI pipeline where all active video processing happens in AXI4-Stream form. The currently implemented processing sequence in this repository is:

RGB → Grayscale → 3x3 Window → Sobel → Overlay/Threshold (1)

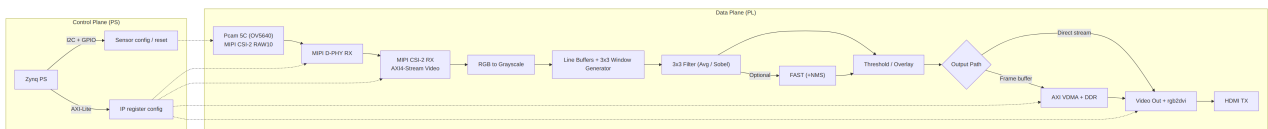


Figure 1: High-level realtime image-processing architecture used throughout this report.

Figure 1 defines the logical data flow boundaries used in RTL and cocotb verification. While the Vivado block design includes additional camera, DDR, and output timing infrastructure, this report focuses on repository-owned modules under `rtl/` and `testbench/`.

2.1. In-Scope Implementations

- AXI4-Stream passthrough and protocol baseline validation.
- Grayscale conversion core and AXI wrapper (RGB_TO_GRAYSCALE).

- Sliding window generation for neighborhood-based filtering (WINDOW_GENERATOR).
- Sobel edge extraction and composition wrapper (SOBEL_FILTER, FILTER_WRAPPER).
- RGB edge overlay composition (EDGE_OVERLAY).
- Button interaction control path (DEBOUNCER, CLICK_DETECTOR, BUTTON_EXAMPLE).
- cocotb target-based regressions and artifact generation ([testbench/sim_build](#)).

2.2. Out-of-Scope in This Report

- Full hardware/software runtime profiling on board.
- Final timing closure and post-route utilization sign-off analysis.
- Software application UX workflows outside this FPGA repository.

The guiding acceptance criterion for all in-scope blocks is AXI4-Stream protocol correctness under both continuous flow and stall conditions.

3. System and Protocol Architecture

Component owner(s): Team-wide integration

The repository follows UG934-compatible AXI4-Stream video semantics with strict sideband interpretation. [1]

- TUSER[0] denotes Start of Frame (SOF).
- TLAST denotes End of Line (EOL).
- Only active pixels are transmitted on AXI4-Stream video.

$$\text{transfer}_k = \text{tvalid}_k \wedge \text{tready}_k \quad (2)$$

The stream contract in Eq. 2 is the basis for all cocotb monitors and scoreboard synchronization.

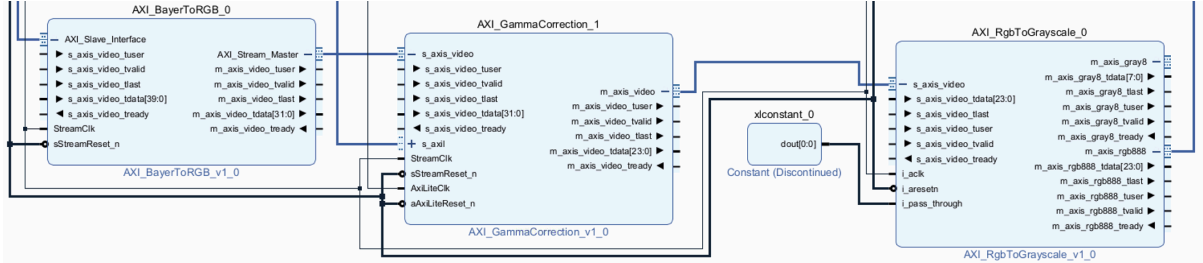


Figure 2: Vivado context around Bayer-to-RGB, gamma correction, and custom grayscale insertion point.

3.1. Timing Domains and Framing Responsibility

The effective architectural split is:

- PS-controlled AXI4-Lite configuration for camera, DMA, timing, and custom IP setup.
- AXI4-Stream compute chain in PL for per-pixel transforms and local neighborhood operations.
- Frame buffering through VDMA/DDR between ingress and display timing domains.

Custom stream modules inside this repository must preserve framing semantics across reset, startup, and backpressure. In practical terms this means that payload, SOF, and EOL all remain beat-aligned whenever TVALID=1, including stalled cycles.

3.2. Framing Recovery Strategy

For modules with internal history (line buffers, sliding windows), frame boundaries require explicit re-initialization logic. The project uses SOF as the event that re-establishes deterministic per-frame state and avoids cross-frame leakage.

AXI4-Stream rule set enforced in this report: 1) accepted transfer iff TVALID && TREADY 2) SOF on first accepted beat of frame 3) EOL on last accepted beat of each line 4) payload/sidebands stable while stalled

4. Verification Framework and Execution Flow

Component owner(s): Lukas Roess, Jan Duchscherer, Valentin Bumeder, Justin Loeber

The verification environment is cocotb-based and target-driven through `tb-sim`. [2] A target in `testbench/targets.toml` declares toplevel HDL, test module, and source set. This keeps DUT selection reproducible and decoupled from command-line complexity.

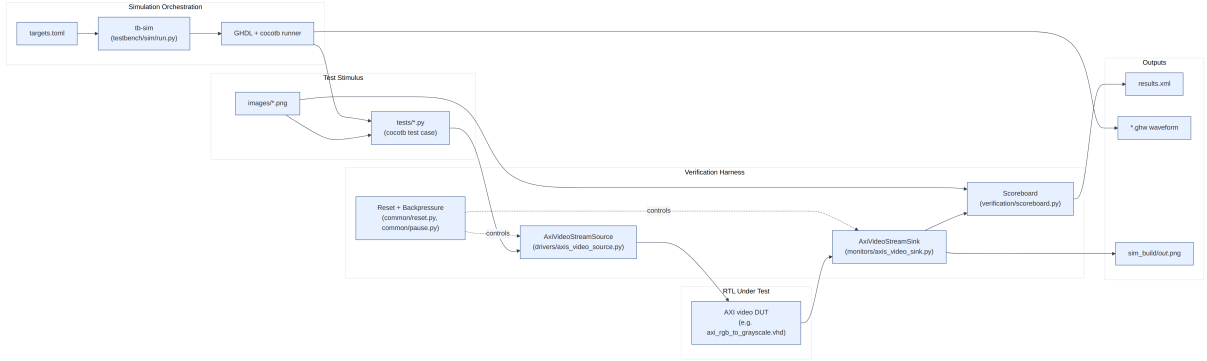


Figure 3: cocotb verification architecture with drivers, monitors, and scoreboard checks.

The testbench combines protocol checks and functional checks using cocotb scheduling/timing semantics and AXI endpoint models.[3], [4]

- Protocol checks: accepted beat counting, stall stability, SOF/EOL semantics.
- Functional checks: expected-vs-observed payload comparisons (pixel stream or window stream).
- Artifact checks: per-run `results.xml`, optional waveforms (`.ghw`), and image outputs.

4.1. Operational Sequence (Streamlit-UI style adaptation)

1) Select simulation target (`tb-sim --target <target-key>`) 2) Resolve sources + toplevel + cocotb module 3) Build and run with simulator backend 4) Drive AXI stream sources under optional backpressure 5) Capture and verify sink outputs in scoreboard 6) Emit XML/wave/image artifacts for reporting

4.2. Verification Targets Used in this Report

Target	DUT focus	Verification intent
example_passthrough	AXI stream pass-through	baseline protocol and roundtrip
axi_rgb_to_grayscale	RGB to gray conversion	functional transform + sideband checks
window_generator	KxK neighborhood streaming	window validity + framing
axi_sobel_filter	edge extraction	gradient behavior + handshake under load
axi_filter_wrapper	window+sobel integrated wrapper	end-to-end streaming integration
test_debouncer/test_click_detector	button path	timing and state transitions

Artifact-only targets observed in stored `sim_build` evidence (but not currently listed in `targets.toml`) are `axi_edge_overlay` and `axi_windowed_filter_wrapper`.

The generated evidence used later in Section 8 is parsed directly from `testbench/sim_build/**/results.xml`.

5. Component Deep Dive: RGB-to-Grayscale

Component owner(s): Jan Duchscherer

The grayscale stage is implemented as a lightweight arithmetic core (`rtl/RGB_TO_GRAYSCALE/hdl/rgb_to_grayscale.vhd`) wrapped by an AXI4-Stream interface module (`rtl/RGB_TO_GRAYSCALE/hdl/axi_rgb_to_grayscale.vhd`). The wrapper preserves handshake and framing while offering both replicated-RGB and gray8 outputs under UG934 framing rules.[1]

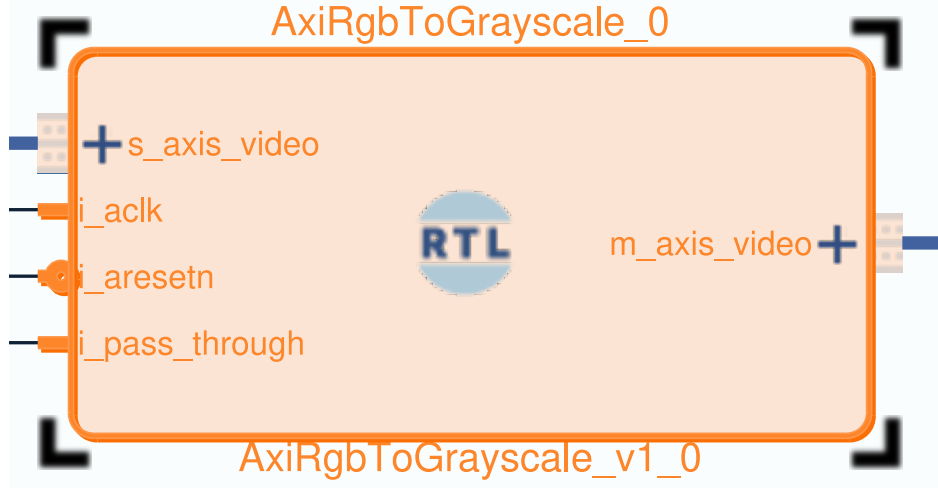


Figure 4: AXI RGB-to-grayscale wrapper and core structure.

5.1. Input/Output Contract

Signal group	Direction	Purpose
s_axis_video_*	Input	RGB stream with AXI handshake + S0F/E0L
m_axis_rgb888_*	Output	RGB-format output for pass-through or replicated gray
m_axis_gray8_*	Output	Single-channel luminance stream for downstream filters
i_pass_through	Input	Selects direct RGB forwarding vs grayscale emit

Important stream-format alignment note: the effective testbench wire order for RGB24 in this project is `TDATA[23:0] = R|B|G`, while Python-side comparisons use `(R,G,B)` tuples after decode.

5.2. Luminance Approximation and Tradeoff

The current arithmetic uses a shift-add approximation to avoid DSP pressure:

$$Y = \left(\frac{R}{4}\right) + \left(\frac{G}{2}\right) + \left(\frac{B}{4}\right) \quad (3)$$

Reference fixed-point alternatives remain compatible when higher luminance fidelity is required.[5]

$$Y = \frac{77R + 150G + 29B + 128}{256} \quad (4)$$

This implementation is intentionally streaming and stateless per pixel; no line buffer is required in this stage.

5.3. Empirical Output Evidence



Figure 5: Left: RGB reference frame. Right: grayscale wrapper output artifact.

Figure 5 confirms expected luminance replication behavior for stored regression artifacts.

5.4. Implementation Notes

- Wrapper tready gating keeps RGB and gray fan-out synchronized.
- Sidebands are forwarded beat-aligned with payload.
- `i_pass_through` enables direct protocol-preserving comparison against input data in tests.

6. Component Deep Dive: Window Generator, Sobel, and Wrapper

Component owner(s): Lukas Roess, Justin Loeber

This stage converts a scalar pixel stream into local neighborhoods and computes gradient-based edges through Sobel kernels before optional thresholding/selection in wrapper logic.

6.1. Window Generator Function

For $K = 3$, each output beat represents a local matrix around the current center pixel:

$$W(x, y) = \begin{pmatrix} p(x-1, y-1) & p(x, y-1) & p(x+1, y-1) \\ p(x-1, y) & p(x, y) & p(x+1, y) \\ p(x-1, y+1) & p(x, y+1) & p(x+1, y+1) \end{pmatrix} \quad (5)$$

Window validity is emitted only after warm-up and border-policy conditions are met. Current tests include no-pressure and backpressure scenarios plus stress patterns.

6.2. Sobel Core Equations

$$G_x = ((p_{02} + 2p_{12} + p_{22}) - (p_{00} + 2p_{10} + p_{20})) \quad (6)$$

$$G_y = ((p_{20} + 2p_{21} + p_{22}) - (p_{00} + 2p_{01} + p_{02})) \quad (7)$$

$$M = |G_x| + |G_y| \quad (8)$$

The wrapper applies threshold selection and stream-level control while preserving AXI framing.

6.3. Pipeline Micro-Flow (Doc-scanner style adaptation)

```
for each accepted input pixel: update line buffers and horizontal taps if warm-up complete:
emit 3x3 window compute Sobel gradients (Gx, Gy) derive edge magnitude M compare against
threshold drive wrapper output beat
```

6.4. Empirical Evidence



Figure 6: Left: Sobel core output artifact. Right: wrapper-level Sobel output artifact.

6.5. Current Engineering Risks in this Block

- Window-generator frame boundary behavior must stay strictly aligned with emitted S0F/EOL.
- Warm-up latency and delayed sidebands require careful reset/restart discipline.
- Wrapper generic combinations should be validated beyond the default 3x3/8-bit operating point.

7. Component Deep Dive: Edge Overlay and Control Path

Component owner(s): Valentin Bumeder, Jan Duchscherer

The overlay stage merges RGB video and binary edge stream in lockstep, while the button/control path handles interactive state transitions through debouncing and click detection.

The shown overlay image evidence is parsed from stored simulation artifacts; the related target is currently treated as artifact-only in reporting because it is not registered in the active [testbench/targets.toml](#) list.

7.1. Edge Overlay Composition

[AxiEdgeOverlay](#) receives two synchronized AXI streams:

- RGB base stream (`s_axis_video_rgb888_*`)
- Edge stream (`s_axis_video_edges_*`)

If `i_overlay_enable = 1` and edge beat is non-zero, the pixel is replaced with `G_EDGE_COLOR`; otherwise the RGB beat is forwarded unchanged.

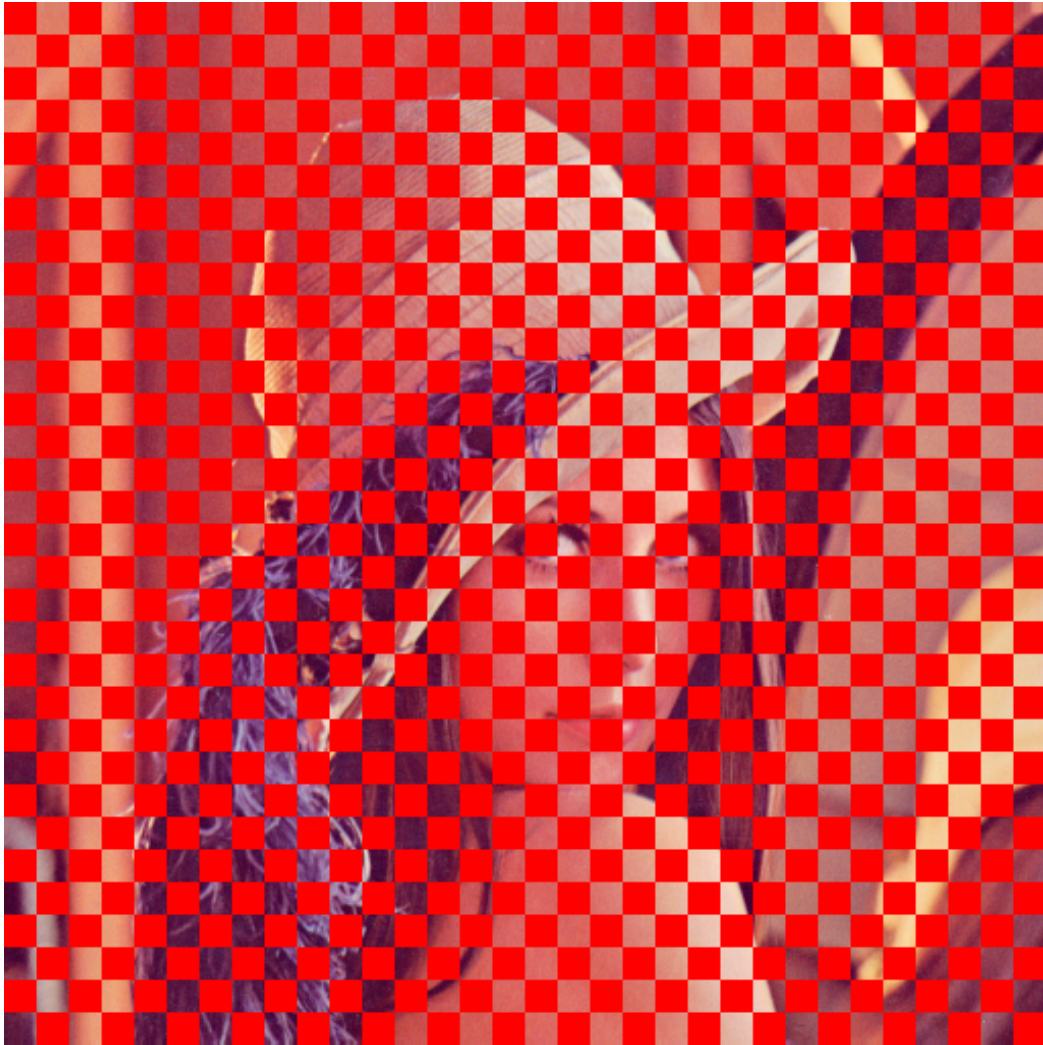


Figure 7: Edge overlay output artifact generated by cocotb testbench.

7.2. Dual-Stream Handshake Policy

```
m_axis_valid = rgb_valid && edge_valid
rgb_ready = m_axis_ready && edge_valid
edge_ready = m_axis_ready && rgb_valid
```

This lockstep handshake prevents one stream from advancing without the other and protects SOF/EOL alignment at merge points.[1]

7.3. Control Path Modules

Module	Role in control path
rtl/DEBOUNCER/hdl/debouncing.vhd	stabilizes asynchronous button transitions and removes chatter
rtl/CLICK_DETECTOR/hdl/click_detection.vhd	classifies click events with state-machine based timing rules
rtl/CLICK_DETECTOR/hdl/debounced_click_detector.vhd	integration wrapper combining debouncer and click FSM
rtl/BUTTON_EXAMPLE/hdl/button_example.vhd	hardware-visible status path used in Vivado integration

These modules are not in the image-processing datapath itself, but they affect usability and real-system operability when switching or enabling stream behaviors. The debouncing assumptions follow standard mechanical switch-bounce behavior summarized in [6].

8. Quantitative Results and Verification Metrics

Component owner(s): Repository-wide regression evidence

This section is generated from `docs/report/data/junit_metrics.csv` and Plotly charts exported by `docs/report/analysis/build_report_assets.py`.

Total testcases

55

Across all discovered results.xml files

Passing

55

Artifact snapshot used by this report

Failing

0

Expected to remain 0 for current baseline

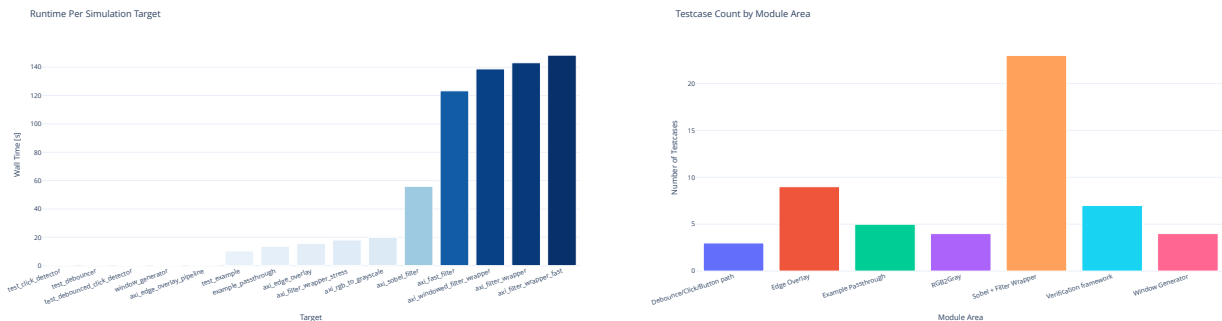


Figure 8: Left: summed wall runtime per target. Right: testcase distribution by module area.

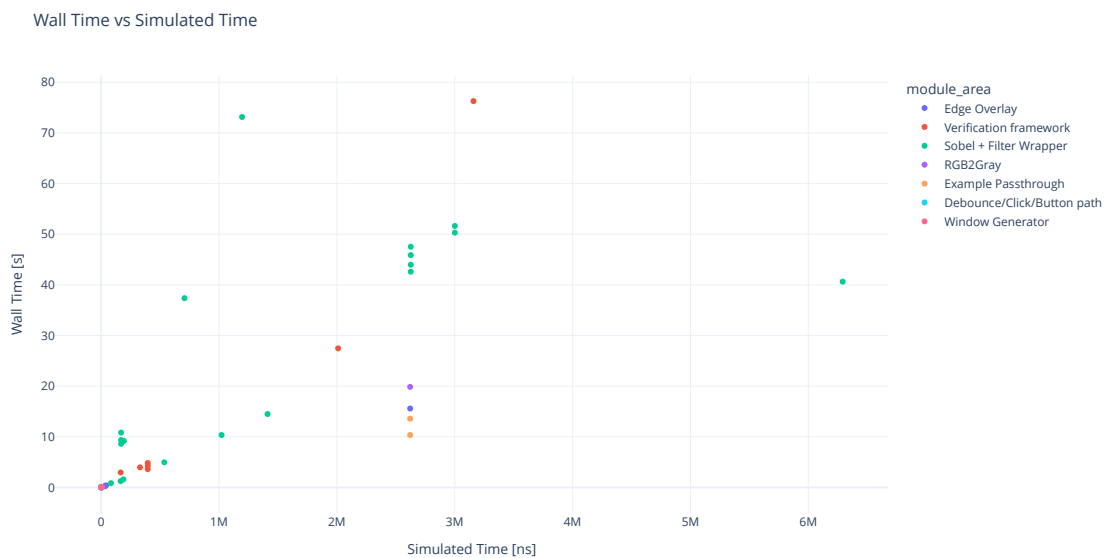


Figure 9: Wall-time versus simulated-time distribution by module area.

8.1. Target Coverage Table

Registry status is evaluated against `testbench/targets.toml`.

Target key	Evidence provenance			
	Recorded count	Registry status		
example_passthrough	3	registered in targets.toml		
axi_rgb_to_grayscale	4	registered in targets.toml		
window_generator	4	registered in targets.toml		
axi_sobel_filter	4	registered in targets.toml		
axi_filter_wrapper	5	registered in targets.toml		
axi_filter_wrapper_stress	5	registered in targets.toml		
axi_edge_overlay	6	artifact-only	target	in stored
axi_windowed_filter_wrapper	3	artifact-only	target	in stored
test_debouncer	1	registered in targets.toml		
test_click_detector	1	registered in targets.toml		

The runtime profile in Figure 8 shows that wrapper-level Sobel regressions dominate wall-clock execution due to end-to-end image and stress scenarios, while pure protocol tests remain lightweight.

9. Team Contributions and Timeline

Component owner(s): All team members

Contribution analytics are generated from canonicalized git history and optional manual overlays. This first draft uses git-derived rows only (`team_contrib_overrides.csv` is intentionally empty).

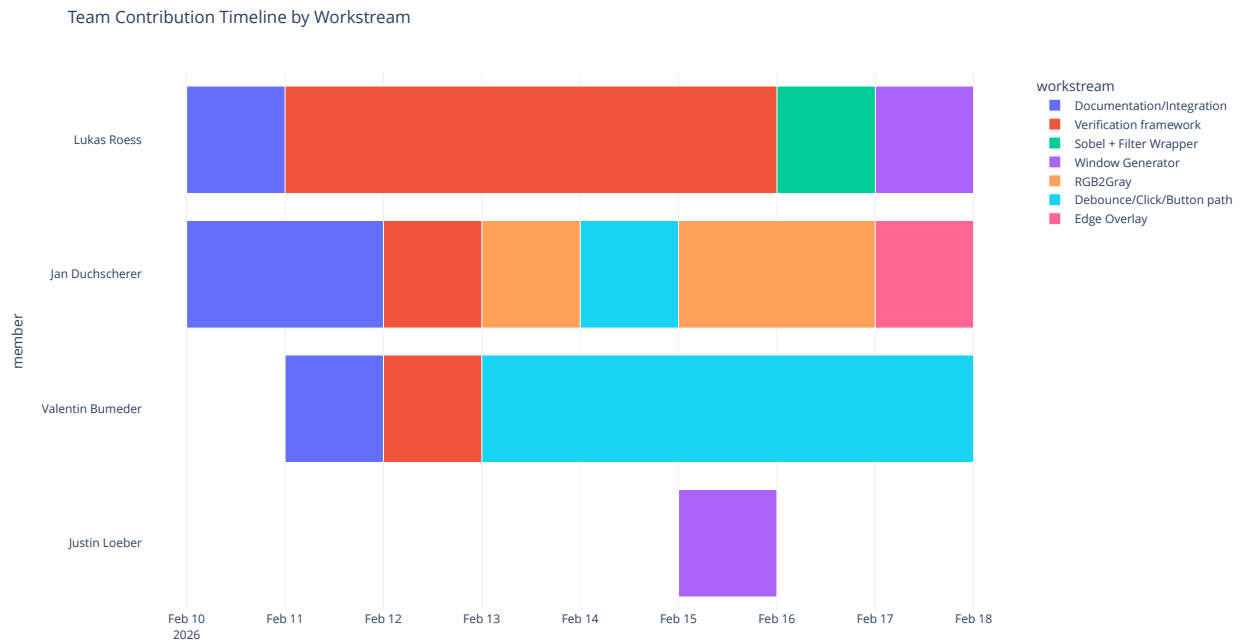


Figure 10: Contribution timeline by team member and workstream (git-derived).

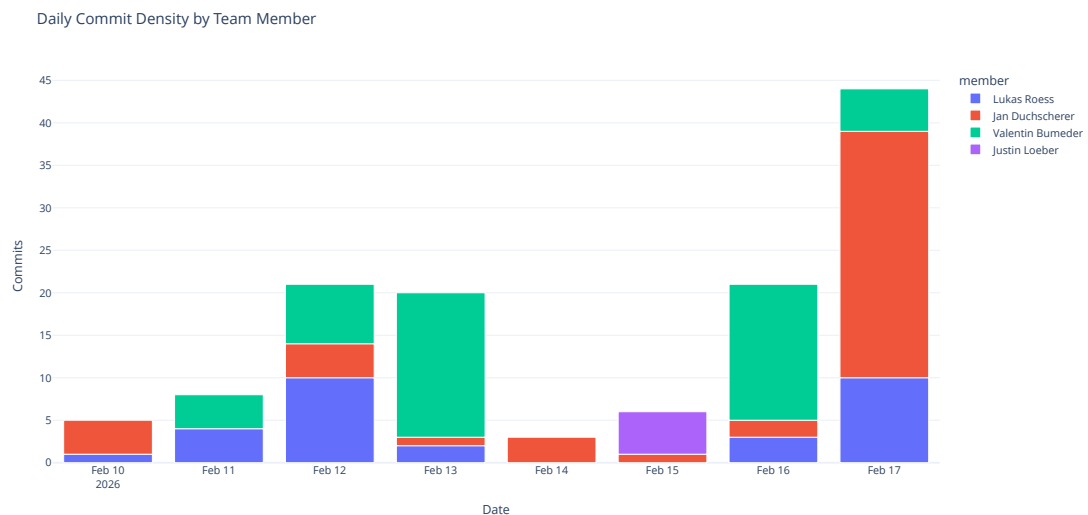


Figure 11: Daily commit density grouped by team member.

Member	Contribution summary		Primary contribution domains
	Git commits	Workstreams	
Lukas Roess	41	4	Verification framework, Window Generator, Sobel + Filter Wrapper, docs integration
Jan Duchscherer	41	5	Documentation/integration, RGB2Gray, verification orchestration, overlay touch points

	Git commits	Workstreams	
Valentin Bumeder	61	3	Debounce/click control path, integration work, verification support
Justin Loeber	6	2	Window Generator feature increments and matching verification support

Interpretation:

- Workstream overlaps confirm parallel development of datapath RTL, verification, and integration docs.
- The timeline in Figure 10 highlights concentrated ownership windows for each component family.
- Commit density in Figure 11 indicates ramp-up around integration and final verification phases.

10. FAST Implementation and FAST-on-Zynq (Zybo Z7-10)

Component owner(s): Implemented RTL path: [rtl/FAST_FILTER/hdl/*](#)

This section documents the implemented FAST pipeline in this repository and summarizes how it maps to the Zybo Z7-10 streaming architecture. The active RTL modules are:

- [rtl/FAST_FILTER/hdl/fast_core.vhd](#)
- [rtl/FAST_FILTER/hdl/fast_nms3x3.vhd](#)
- [rtl/FAST_FILTER/hdl/axi_fast_filter.vhd](#)

The implementation uses a FAST-9 style contiguous arc test on a 7x7 window, computes a corner score, and applies 3x3 non-maximum suppression before emitting a binary corner mask on AXI.[7], [8]

10.1. FAST Corner Detector Fundamentals

FAST (Features from Accelerated Segment Test) classifies a pixel center p as corner when a contiguous arc on the 16-point circle around p is consistently brighter or darker than the center by threshold t . [7], [8]

A center pixel is a FAST corner when a contiguous subset S on the 16-point circle satisfies the run-length condition:

$$|S| \geq N \quad (9)$$

and each pixel in that run passes either the bright or dark threshold test:

$$I_q \geq I_p + t \quad (10)$$

$$I_q \leq I_p - t \quad (11)$$

Where:

- C_{16} is the 16-point circle (radius 3) around the center pixel.
- N is typically 9, 10, or 12 (FAST-9 is common in realtime settings).
- t controls corner selectivity and noise sensitivity.

The implementation also uses the classic high-speed precheck (4 ring points) before full run evaluation.[7]

10.2. Typst Pseudocode: [E_FastCore](#) (Candidate + Score)

Pseudocode in Algorithm 1 summarizes the combinational FSM behavior in [rtl/FAST_FILTER/hdl/fast_core.vhd](#).

Algorithm 1: FAST core pseudocode aligned with [E_FastCore](#)

```

1: procedure FASTCORE7X7(window7x7, threshold, N)
2:   center ← pixel(window7x7, 24)
3:   T ← clamp(threshold, 0, 255)
4:   high_ref ← center + T
5:   low_ref ← center - T

```

```

6:   ring[0..15] ← window7x7[C_RING_INDEX[i]]
7:   bright_pre ← count(ring[p] > high_ref for p in {0,8,4,12})
8:   dark_pre ← count(ring[p] < low_ref for p in {0,8,4,12})
9:   if bright_pre < 3 and dark_pre < 3 then
10:    return (is_candidate=0, score=0)
11:  end
12:  best_score ← 0
13:  is_candidate ← 0
14:  for start_idx in 0..15 do
15:    bright_run ← true
16:    dark_run ← true
17:    bright_score ← 0
18:    dark_score ← 0
19:    for off_idx in 0..N-1 do
20:      idx ← (start_idx + off_idx) mod 16
21:      value ← ring[idx]
22:      if value > high_ref then
23:        bright_score ← bright_score + (value - high_ref)
24:      end
25:      if value <= high_ref then
26:        bright_run ← false
27:      end
28:      if value < low_ref then
29:        dark_score ← dark_score + (low_ref - value)
30:      end
31:      if value >= low_ref then
32:        dark_run ← false
33:      end
34:    end
35:    if bright_run then
36:      is_candidate ← 1
37:      best_score ← max(best_score, bright_score)
38:    end
39:    if dark_run then
40:      is_candidate ← 1
41:      best_score ← max(best_score, dark_score)
42:    end
43:  end
44:  best_score ← min(best_score, 8191)
45:  return (is_candidate, best_score)
46: end

```

10.3. Typst Pseudocode: **AXI_FastFilter** (Core + NMS + AXI)

Pseudocode in Algorithm 2 captures the stream-level composition in `rtl/FAST_FILTER/hdl/axi_fast_filter.vhd`.

Algorithm 2: AXI FAST filter pipeline pseudocode aligned with **AXI_FastFilter**

```

1: procedure AXIFASTFILTER(s_axis_window, m_axis_filter8)
2:   ▷ Per accepted 7x7 input window
3:   (is_candidate, score13) <- FastCore7x7(window, G_FAST_THRESHOLD, G_FAST_N)
4:   score_pixel ← score13 if is_candidate else 0
5:   ▷ Build 3x3 neighborhoods in score domain
6:   score_window3x3 <- WindowGenerator3x3(score_pixel)
7:   ▷ Non-maximum suppression
8:   corner ← FastNms3x3(score_window3x3)
9:   ▷ Emit binary corner mask
10:  out_pixel ← 0xFF if corner else 0x00
11:  ▷ AXI sidebands forwarded with reset-safe gating
12:  TVALID/TREADY/TUSER/TLAST follow score-window output timing

```

10.4. Streaming Hardware Mapping

The current design chain for FAST is:

- Gray window input (7x7) from window_generator
- FAST candidate/score extraction ([E_FastCore](#))
- Score-window generation (3x3) for NMS
- NMS corner decision ([E_FastNms3x3](#))
- AXI output mask (0xFF corner, 0x00 non-corner)

This keeps FAST fully streaming and compatible with the existing AXI framing discipline.

10.5. FAST on Zybo Z7-10 (Zynq-7010) Integration Strategy

The Zybo Z7-10 project already uses PS7 + AXI4-Stream video infrastructure and VDMA buffering. FAST is integrated as an AXI4-Stream block and selected through wrapper-level filter configuration.

Module role	FAST-on-Zybo implementation status
Input format	Gray stream windows are consumed as 7x7 flattened AXI payloads.
Decision core	E_FastCore implements precheck + contiguous-run FAST test and score saturation.
Post-processing	E_FastNms3x3 suppresses non-maxima in local 3x3 score neighborhoods.
System interface	AXI_FastFilter forwards AXI handshakes and framing with reset-safe gating.
Verification	Targets <code>axi_fast_filter</code> and <code>axi_filter_wrapper_fast</code> are present in testbench/targets.toml .

10.6. Throughput and Latency Expectations

For a single-pixel-per-cycle (PPC=1) pipeline, ideal frame-rate bound is:

$$F = \frac{f}{W * H} \quad (12)$$

where f is the processing clock frequency in pixels per second.

At $f_{clk} = 100$ MHz:

- 512x512: about 381 FPS theoretical upper bound.
- 1280x720: about 108 FPS theoretical upper bound.

Practical results are lower due to buffering, downstream backpressure, and NMS latency.

10.7. Risk and Validation Plan for FAST Path

- Threshold sensitivity: fixed threshold may underperform across lighting changes; mitigation is runtime-configurable `G_FAST_THRESHOLD` and sweep tests.
- Border behavior: radius-3 circle implies larger invalid margins than Sobel; mitigation is explicit border policy mirrored in `cocotb` golden model.
- Timing closure risk: 16-point comparison + contiguous run logic can stretch critical paths; mitigation is staged combinational partitioning and scoreboard-backed regressions.
- Integration risk: preserving AXI S0F/E0L across 7x7 warm-up and score-domain NMS must be continuously validated under backpressure.

The implemented FAST path now complements the Sobel path in the same wrapper architecture and enables direct algorithm-level comparison on identical streaming infrastructure.

11. Risks, Limitations, and Next Steps

Component owner(s): Engineering review

The current implementation baseline is functional for repository-level verification artifacts, but full deployment readiness requires additional closure steps.

Risk	Impact	Mitigation
Window-generator framing drift under corner reset scenarios	High	Add dedicated multi-frame stress vectors with explicit SOF restart assertions and waveform review gates.
Wrapper generic combinations under-exercised	Medium	Expand target matrix for resolution/kernel/threshold variations and include automatic parameter sweep summaries.
Legacy artifact targets not registered in active target list	Medium	Either re-register <code>axi_edge_overlay/axi_windowed_filter_wrapper</code> in <code>targets.toml</code> or exclude them from current-baseline KPI calculations.
Limited board-level performance evidence	Medium	Add hardware runbook with timing/utilization snapshots and in-system throughput counters.
Manual override contribution data currently empty	Low	Enable structured non-commit effort entries in <code>team_contrib_overrides.csv</code> for final report revision.

11.1. Limitations of the Current Evidence Set

- Results are derived from stored simulation artifacts, not a freshly re-run full regression in this report build.
- The report currently emphasizes functional correctness and protocol behavior more than synthesis resource economics.
- Some historical artifacts reference legacy target names; these are parsed and retained for traceability.

11.2. Priority Follow-Up Actions

1. Expand stress scenarios for window and wrapper paths at additional frame sizes.
2. Add synthesis/implementation metrics (LUT/FF/BRAM/Fmax) in a dedicated quantitative appendix.
3. Integrate board-level runtime validation with camera live feed and overlay switching controls.
4. Populate manual contribution overrides where design reviews or offline debugging were significant.

12. Conclusion

This repository now contains a coherent RTL and verification baseline for a real-time FPGA image-processing pipeline with AXI4-Stream compliance as the central engineering constraint. The implemented chain spans grayscale conversion, local window generation, Sobel edge extraction, wrapper integration, and edge overlay, with auxiliary button-control logic for system interaction.

The generated metrics and artifacts demonstrate broad testcase coverage and no recorded failures in the parsed dataset. Equally important, the contribution timeline shows coordinated parallel development across algorithmic modules, verification infrastructure, and integration documentation.

From an implementation perspective, the project has transitioned from isolated IP work toward an end-to-end stream pipeline that is testable, inspectable, and extensible. The next milestone is to close the gap between simulation-grade confidence and production-grade deployment through expanded stress matrices, board-level validation evidence, and final timing/resource closure.

References

- [1] “AXI4-Stream Video IP and System Design Guide (UG934).” Nov. 2022. Accessed: Feb. 17, 2026. [Online]. Available: https://docs.amd.com/r/en-US/ug934_axi_videoIP
- [2] cocotb contributors, “Writing Testbenches.” Accessed: Feb. 17, 2026. [Online]. Available: https://docs.cocotb.org/en/stable/writing_testbenches.html
- [3] cocotb contributors, “The Timing Model.” Accessed: Feb. 17, 2026. [Online]. Available: https://docs.cocotb.org/en/stable/timing_model.html
- [4] A. Forencich, “cocotbext-axi.” Accessed: Feb. 17, 2026. [Online]. Available: <https://github.com/alexforencich/cocotbext-axi>
- [5] “Recommendation ITU-R BT.601-7: Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios,” technical report BT.601-7, Mar. 2011. Accessed: Feb. 17, 2026. [Online]. Available: <https://www.itu.int/rec/R-REC-BT.601-7-201103-I/en>
- [6] J. Ganssle, “A Guide to Debouncing.” Accessed: Feb. 17, 2026. [Online]. Available: <https://www.ganssle.com/debouncing.htm>
- [7] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006, pp. 430–443. doi: 10.1007/11744023_34.
- [8] E. Rosten, R. Porter, and T. Drummond, “Faster and Better: A Machine Learning Approach to Corner Detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 32, no. 1, pp. 105–119, 2010, doi: 10.1109/TPAMI.2008.275.